FIAP – PÓS TECH

PÓS-GRADUAÇÃO EM DEVOPS E ARQUITETURA CLOUD

TECH CHALLENGE – FASE 2

PROJETO TOGGLEMASTER

NOME:

Flávio Alan Nascimento Amâncio

RM376243

Usuário Discord: flavioamancio8500

Github: https://github.com/FlavioAmancio/togglemaster-fase2.git

Barra do Piraí – Rio de Janeiro
2026

Relatório Técnico:

Fase 1 – Conteinerização e Execução Local (ToggleMaster)

1. Visão Geral e Arquitetura Local

O objetivo desta primeira etapa foi realizar a transição da arquitetura monolítica para um ecossistema distribuído com **5 microsserviços**, orquestrados localmente via Docker e Docker Compose junto a seus bancos de dados e caches dedicados.

Conforme o requisito do desafio, o ambiente local necessita de **9 contêineres** operando simultaneamente e com comunicação íntegra:

Serviço / Contêiner	Tipo / Stack	Porta Local	Responsabilidade / Armazenamento
auth-service	Go (v1.22)	8001	Autenticação e emissão/validação de API Keys.
flag-service	Python / Flask	8002	CRUD e regras de ativação das feature flags.
targeting-service	Python / Flask	8003	Regras complexas de segmentação (targeting).
evaluation-service	Go (v1.22)	8004	Hot path de alta performance (avaliação de flags).
analytics-service	Python / Flask	8005	Worker assíncrono para ingestão de métricas.
postgres-auth	PostgreSQL 15	5432	Banco isolado para o serviço de autenticação (auth_db).
postgres-app	PostgreSQL 15	5433	Banco contendo flags_db e targeting_db.
redis	Redis 7	6379	Cache de baixa latência para o evaluation-service.
dynamodb-local	Amazon DynamoDB	8000	Banco NoSQL local emulado para o analytics-service.

2. Decisões de Engenharia na Conteinerização (Dockerfiles)

Para garantir segurança, baixo consumo de recursos e imagens otimizadas para produção, foram adotadas práticas recomendadas de mercado:

A. Microsserviços em Go (auth-service e evaluation-service)

Foi empregado o padrão **Multi-Stage Build**:

1. **Estágio de Build (golang:1.22-alpine):** Ambiente completo com compilador. O binário estático foi gerado com flags de redução de tamanho (-ldflags="-w -s"), que removem tabelas de símbolos e informações de debug.

2. **Estágio de Execução (alpine:3.19):** Imagem final enxuta, sem o compilador Go ou ferramentas desnecessárias, reduzindo a superfície de ataque e o tamanho da imagem de centenas de megabytes para cerca de **15 MB a 25 MB**.

3. **Instalação de Certificados TLS:** Foi adicionado o pacote ca-certificates explicitamente no Alpine para que as chamadas HTTPS para a AWS (como o SQS) não sofram falhas de certificado SSL/TLS em tempo de execução.

B. Microsserviços em Python (flag-service, targeting-service, analytics-service)

1. **Imagem Base Leve:** Utilização do python:3.11-slim para evitar o excesso de pacotes desnecessários da imagem completa do Debian.

2. **Otimizações de Runtime Python:**

- PYTHONDONTWRITEBYTECODE=1: Evita a gravação de arquivos .pyc dentro do contêiner.

- PYTHONUNBUFFERED=1: Força a saída dos logs em tempo real para o stdout/stderr do Docker, fundamental para monitoramento e observabilidade.

3. **Servidor de Produção WSGI:** Utilização do gunicorn com binding dinâmico para a porta exposta de cada microsserviço.

3. Gestão dos Bancos de Dados Locais e Automação

O enunciado exigiu explicitamente **4 bancos de dados locais (2 instâncias PostgreSQL, 1 Redis e 1 DynamoDB Local)**:

1. **Separação de Instâncias Postgres:**

- Para simular o isolamento que existirá na nuvem (RDS), o auth-service conectou-se a uma instância exclusiva (postgres-auth na porta do host 5432), executando seu próprio db/init.sql.

- Os serviços flag-service e targeting-service compartilham a segunda instância (postgres-app na porta do host 5433).

2. **Script de Inicialização Múltipla (init-app-dbs.sh):**

- O contêiner padrão do PostgreSQL suporta apenas um banco via variáveis de ambiente (POSTGRES_DB).

- Para provisionar tanto o flags_db quanto o targeting_db automaticamente sem intervenção manual, foi criado o script init-app-dbs.sh, montado no diretório /docker-entrypoint-initdb.d/.

- Esse script cria as duas bases e já executa os respectivos schemas (flag-init.sql e targeting-init.sql).

4. Desafios Encontrados, Diagnósticos e Correções Cirúrgicas

Durante a execução do ambiente local, foram detectados e resolvidos **6 problemas críticos** originados de código legado, dependências desatualizadas e configurações do compose:

Desafio 1: Injeção de Variáveis Não Declaradas e Atributo Obsoleto no Docker Compose

- **Sintoma:** O Docker emitia diversos alertas no terminal:

```
WARN[0000] The "AWS_ACCESS_KEY_ID" variable is not set. Defaulting to a blank string.
WARN[0000] the attribute `version` is obsolete...
```

- **Causa Raiz:** O Compose continha referências com ${AWS_ACCESS_KEY_ID}, buscando variáveis no sistema operacional local do host, além da tag version: '3.8' (descontinuada nas versões recentes do Docker Compose v2).

- **Solução:**

1. Remoção da linha version: '3.8'.

2. Definição de credenciais locais estáticas e fictícias (mock_key_local, etc.) diretamente no arquivo, garantindo operação 100% isolada e sem avisos no terminal.

Desafio 2: Erro de Sintaxe no Módulo Go (auth-service/go.mod)

- **Sintoma:** Falha na compilação do contêiner com as seguintes mensagens sucessivas do Go:

```
go: errors parsing go.mod:
go.mod:18:2: require github.com/jackc/pgx/v4/stdlib: version "v4.18.3" invalid: should be v0 or v1, not v4
go.mod:18:2: usage: require module/path v1.2.3
```

- **Causa Raiz:** No arquivo go.mod original, a dependência do driver PostgreSQL foi declarada de maneira incorreta:

1. Primeiro, tentava importar o subpacote /stdlib como um módulo raiz versionado em v4.

2. Na linha 18, havia a declaração corrompida: github.com/jackc/pgx/v4/v4.18.3 // indirect, onde a versão estava colada ao nome do pacote com uma barra (/) e sem versão declarada.

- **Solução Aplicada em auth-service/go.mod:**

➓ Correção do bloco require para o módulo raiz:

```
require (
    github.com/jackc/pgx/v4 v4.18.3
    github.com/joho/godotenv v1.5.1
)
```

➓ Remoção da linha duplicada e corrompida do bloco require (...) // indirect.

Desafio 3: Restrições Estritas de Compilação do Go (Imports Não Utilizados e Driver SQL)

- **Sintoma:** O comando go build falhou com:

```
./handlers.go:4:2: "crypto/sha256" imported and not used
./handlers.go:5:2: "encoding/hex" imported and not used
./key.go:7:2: "fmt" imported and not used
./main.go:5:2: "fmt" imported and not used
./main.go:10:2: "github.com/jackc/pgx/v4/stdlib" imported and not used
```

- **Causa Raiz:** O compilador Go bloqueia a compilação por padrão caso existam pacotes importados que não são chamados no código. Além disso, em main.go, o driver de banco precisava ser importado exclusivamente por seus efeitos colaterais de registro (via identificador anônimo _).

- **Solução Aplicada em auth-service:**

➓ handlers.go: Removidos os imports "crypto/sha256" e "encoding/hex".

➓ key.go: Removido o import "fmt".

➓ main.go: Removido o import "fmt" e adicionado o identificador anônimo ao driver:

```
_ "github.com/jackc/pgx/v4/stdlib"
```

Desafio 4: Pacote Ausente e Import Não Utilizado no evaluation-service

- **Sintoma:** Falha na compilação do evaluation-service:

./evaluator.go:4:2: "context" imported and not used
./evaluator.go:106:12: undefined: os
./evaluator.go:133:12: undefined: os

- **Causa Raiz:** O arquivo importava "context" sem utilizá-lo, mas realizava leituras do sistema operacional (os.Getenv(...)) nas linhas 106 e 133 sem ter o pacote "os" importado.

- **Solução Aplicada em evaluation-service/evaluator.go:**

➓ Remoção de "context".

➓ Adição de "os" no cabeçalho import (...).

Desafio 5: Incompatibilidade entre Flask e Werkzeug nos Serviços Python

- **Sintoma:** Imediatamente após subirem, os contêineres flag-service, targeting-service e analytics-service finalizavam com status Exited (3) e a seguinte exceção nos logs:

ImportError: cannot import name 'url_quote' from 'werkzeug.urls'

- **Causa Raiz:** O requirements.txt instalava uma versão legada do framework Flask sem travar a versão da sua dependência primária (Werkzeug). Ao rodar o build, o pip baixava a versão mais recente do Werkzeug (3.x), na qual o método legado url_quote foi permanentemente removido, quebrando a inicialização do Flask.

- **Solução Aplicada nos 3 Dockerfiles:**

➓ Fixação forçada da biblioteca para uma versão compatível (< 3.0):

RUN pip install --no-cache-dir -r requirements.txt && pip install --no-cache-dir "werkzeug<3.0"

Desafio 6: Parâmetro Incorreto no DynamoDB Local

- **Sintoma:** O contêiner dynamodb-local finalizava com status Exited (1) apresentando o erro:

Error: Unable to access jarfile ApacheDynamoDBLocal.jar

- **Causa Raiz:** O arquivo .jar na imagem oficial distribuída pela AWS (amazon/dynamodb-local:latest) chama-se DynamoDBLocal.jar, e o comando inicial tentava chamar ApacheDynamoDBLocal.jar.

- **Solução Aplicada no docker-compose.yml:**

➓ Correção do comando:

command: "-jar DynamoDBLocal.jar -inMemory -sharedDb"

5. Validação e Teste de Aceitação da Fase 1

Após a aplicação de todas as correções de arquitetura e código, foram realizados dois testes formais de aceitação:

Teste 1: Estado dos Contêineres (docker compose ps)

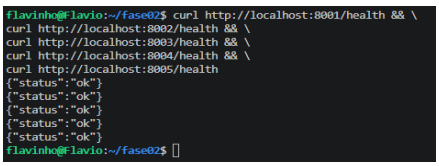
Todos os 9 contêineres permaneceram em execução contínua e saudáveis:



```
flavinho@flavio:~/fase02$ docker compose ps
NAME                IMAGE                          COMMAND                  SERVICE             CREATED          STATUS                    PORTS
analytics-service   fase02-analytics-service       "gunicorn --bind 0.0…"   analytics-service   22 seconds ago   Up 20 seconds             0.0.0.0:8005->8005/tcp, [::]:8005->8005/tcp
auth-service        fase02-auth-service            "./auth-service"         auth-service        22 seconds ago   Up 10 seconds             0.0.0.0:8001->8001/tcp, [::]:8001->8001/tcp
dynamodb-local      amazon/dynamodb-local:latest   "java -jar DynamoDBL…"   dynamodb-local      22 seconds ago   Up 21 seconds             0.0.0.0:8000->8000/tcp, [::]:8000->8000/tcp
evaluation-service  fase02-evaluation-service      "./evaluation-service"   evaluation-service  22 seconds ago   Up 9 seconds              0.0.0.0:8004->8004/tcp, [::]:8004->8004/tcp
flag-service        fase02-flag-service            "gunicorn --bind 0.0…"   flag-service        22 seconds ago   Up 9 seconds              0.0.0.0:8002->8002/tcp, [::]:8002->8002/tcp
postgres-app        postgres:15-alpine             "docker-entrypoint.s…"   postgres-app        22 seconds ago   Up 21 seconds (healthy)   0.0.0.0:5433->5432/tcp, [::]:5433->5432/tcp
postgres-auth       postgres:15-alpine             "docker-entrypoint.s…"   postgres-auth       22 seconds ago   Up 21 seconds (healthy)   0.0.0.0:5432->5432/tcp, [::]:5432->5432/tcp
redis               redis:7-alpine                 "docker-entrypoint.s…"   redis               22 seconds ago   Up 21 seconds (healthy)   0.0.0.0:6379->6379/tcp, [::]:6379->6379/tcp
targeting-service   fase02-targeting-service       "gunicorn --bind 0.0…"   targeting-service   22 seconds ago   Up 9 seconds              0.0.0.0:8003->8003/tcp, [::]:8003->8003/tcp
flavinho@flavio:~/fase02$
```

Teste 2: Health Checks Unificados

Chamada concorrente a todas as rotas /health:




```
curl http://localhost:8001/health && \
curl http://localhost:8002/health && \
curl http://localhost:8003/health && \
curl http://localhost:8004/health && \
  curl http://localhost:8005/health
```

Resultado Obtido:

```
{"status":"ok"}
{"status":"ok"}
{"status":"ok"}
{"status":"ok"}
{"status":"ok"}
```

Conclusão da Fase 1: O ecossistema está validado, robusto e apto a ser transposto para a nuvem AWS na Fase 2.

2. Fase

2: Provisionamento de Infraestrutura na Nuvem (AWS Academy)

2.1 Elastic Container Registry (ECR)

- **Objetivo:** Hospedar as imagens Docker dos 5 microsserviços em um registro privado e seguro na nuvem da AWS, permitindo que o cluster Kubernetes faça o download (pull) das imagens.

- **Arquivo Criado:** push-to-ecr.sh

- **Por que foi criado?** Para evitar a execução manual e repetitiva de 15 comandos no terminal (login, criação de 5 repositórios, 5 tags e 5 pushes), além de tornar o processo idempotente e dinâmico.

Código do Script (push-to-ecr.sh):

```
#!/bin/bash

# Encerra o script imediatamente se algum comando falhar
set -e

# 1. DEFINIÇÃO DE VARIÁVEIS
# Busca automaticamente o ID da sua conta AWS conectada no momento (não precisa deixar fixo!)
ACCOUNT_ID=$(aws sts get-caller-identity --query Account --output text)
REGION="us-east-1"
ECR_URL="${ACCOUNT_ID}.dkr.ecr.${REGION}.amazonaws.com"

echo "=================================================="
echo "Iniciando processo para a Conta AWS: ${ACCOUNT_ID}"
echo "Região: ${REGION}"
echo "URL do ECR: ${ECR_URL}"
echo "=================================================="

# 2. AUTENTICAÇÃO DO DOCKER NA AWS
echo "1/3 -> Autenticando o Docker no Amazon ECR..."
aws ecr get-login-password --region ${REGION} | docker login --username AWS --password-stdin ${ECR_URL}

# 3. LISTA COM O NOME DOS 5 MICROSSERVIÇOS
SERVICES=("auth-service" "flag-service" "targeting-service" "evaluation-service" "analytics-service")

# 4. LOOP DE CRIAÇÃO, TAG E PUSH
for SERVICE in "${SERVICES[@]}"; do
  echo ""
  echo "--------------------------------------------------"
  echo "□ Processando: ${SERVICE}"
  echo "--------------------------------------------------"

  # Cria o repositório no ECR caso ele ainda não exista
  # O '|| true' no final garante que se o repositório já existir, o script não trava com erro
  aws ecr create-repository --repository-name ${SERVICE} --region ${REGION} 2>/dev/null || echo "Repositório ${SERVICE} já existe, continuando..."
```

```
  # Aplica o endereço da AWS como 'apelido' (tag) na imagem que compilamos na Etapa 1
  echo "Etiquetando (tagging) imagem local..."
  docker tag fase02-${SERVICE}:latest ${ECR_URL}/${SERVICE}:latest

  # Envia as camadas da imagem para os servidores da AWS
  echo "Enviando (pushing) imagem para o ECR..."
  docker push ${ECR_URL}/${SERVICE}:latest
done

echo ""
echo "================================================="
echo "✅SUCESSO: Todas as 5 imagens estão no Amazon ECR!"
echo "================================================="
```

2.2 Provisionamento dos Bancos de Dados e Mensageria

- **Objetivo:** Provisionar os recursos gerenciados de persistência e mensageria exigidos pela arquitetura distribuída:

1. **SQS (Simple Queue Service):** Fila padrão togglemaster-events para desacoplar a ingestão de eventos.
2. **DynamoDB:** Tabela NoSQL ToggleMasterAnalytics (Chave Primária: event_id do tipo String) para armazenar eventos de análise.
3. **ElastiCache (Redis):** Cluster em memória evaluation-cache (nó cache.t3.micro) para cache do hot path.
4. **RDS PostgreSQL:** 3 instâncias de banco independentes (auth-db, flag-db, targeting-db), classe db.t3.micro.

- **Arquivo Criado:** provision-databases.sh

Código do Script (provision-databases.sh):

```
#!/bin/bash

# Encerra caso ocorra erro grave inesperado
set -e

# ==============================================================================
# CONFIGURAÇÕES GERAIS
# ==============================================================================
REGION="us-east-1"
DB_PASS="ToggleMaster2024!"  # Senha forte padrão para os 3 bancos RDS

echo "================================================================="
echo "□ INICIANDO PROVISIONAMENTO DE DADOS: REDIS & RDS POSTGRESQL"
echo "================================================================="

# ==============================================================================
# 1. MAPEAMENTO DE REDE (VPC E SUBNETS)
# ==============================================================================
# Descobre dinamicamente a VPC Padrão da sua conta AWS Academy e seu Security Group
echo "1/4 -> Coletando informações de rede da VPC Padrão..."
VPC_ID=$(aws ec2 describe-vpcs --filters "Name=isDefault,Values=true" --query "Vpcs[0].VpcId" --output text --region ${REGION})
SG_ID=$(aws ec2 describe-security-groups --filters "Name=vpc-id,Values=${VPC_ID}" "Name=group-name,Values=default" --query
"SecurityGroups[0].GroupId" --output text --region ${REGION})
SUBNET_IDS=$(aws ec2 describe-subnets --filters "Name=vpc-id,Values=${VPC_ID}" --query "Subnets[*].SubnetId" --output text --
region ${REGION})

echo "      VPC identificada: ${VPC_ID}"
echo "      Security Group identificado: ${SG_ID}"

# ==============================================================================
# 2. LIBERAÇÃO DE PORTAS DE COMUNICAÇÃO
# ==============================================================================
# Abre as portas necessárias para que o Kubernetes e você consigam falar com os bancos
echo "2/4 -> Liberando portas de acesso no Firewall (Security Group)..."
# Porta 5432: Padrão mundial do PostgreSQL
aws ec2 authorize-security-group-ingress --group-id ${SG_ID} --protocol tcp --port 5432 --cidr 0.0.0.0/0 --region ${REGION} 2>/dev/null
|| echo "      Porta 5432 já estava liberada."
# Porta 6379: Padrão mundial do Redis
aws ec2 authorize-security-group-ingress --group-id ${SG_ID} --protocol tcp --port 6379 --cidr 0.0.0.0/0 --region ${REGION} 2>/dev/null
|| echo "      Porta 6379 já estava liberada."

# ==============================================================================
# 3. PROVISIONAMENTO DO REDIS (ELASTICACHE)
```

```
# ============================================================================
# O ElastiCache exige um 'Subnet Group' (grupo de sub-redes) antes de criar o cluster
echo "3/4 -> Solicitando criação do Cluster Redis (ElastiCache)..."
aws elasticache create-cache-subnet-group \
  --cache-subnet-group-name "togglemaster-cache-subnet" \
  --cache-subnet-group-description "Subnet group para o cluster Redis ToggleMaster" \
  --subnet-ids ${SUBNET_IDS} \
  --region ${REGION} 2>/dev/null || echo "      Subnet Group já existe."

# Cria o nó de cache Redis (tamanho t3.micro, o mais econômico)
aws elasticache create-cache-cluster \
  --cache-cluster-id "evaluation-cache" \
  --engine redis \
  --cache-node-type cache.t3.micro \
  --num-cache-nodes 1 \
  --cache-subnet-group-name "togglemaster-cache-subnet" \
  --security-group-ids ${SG_ID} \
  --region ${REGION} 2>/dev/null || echo "      Cluster Redis já solicitado anteriormente."

# ============================================================================
# 4. PROVISIONAMENTO DAS 3 INSTÂNCIAS POSTGRESQL (RDS)
# ============================================================================
echo "4/4 -> Solicitando criação das 3 instâncias de banco RDS PostgreSQL..."

# 4.1 Banco do auth-service
echo "      -> Criando banco: auth-db..."
aws rds create-db-instance \
  --db-instance-identifier auth-db \
  --db-name auth_db \
  --engine postgres \
  --engine-version 15 \
  --db-instance-class db.t3.micro \
  --allocated-storage 20 \
  --master-username postgres \
  --master-user-password "${DB_PASS}" \
  --publicly-accessible \
  --vpc-security-group-ids ${SG_ID} \
  --backup-retention-period 0 \
  --no-multi-az \
  --region ${REGION} 2>/dev/null || echo "      auth-db já solicitado anteriormente."

# 4.2 Banco do flag-service
echo "      -> Criando banco: flag-db..."
aws rds create-db-instance \
  --db-instance-identifier flag-db \
  --db-name flags_db \
  --engine postgres \
  --engine-version 15 \
  --db-instance-class db.t3.micro \
  --allocated-storage 20 \
  --master-username postgres \
  --master-user-password "${DB_PASS}" \
  --publicly-accessible \
  --vpc-security-group-ids ${SG_ID} \
  --backup-retention-period 0 \
  --no-multi-az \
  --region ${REGION} 2>/dev/null || echo "      flag-db já solicitado anteriormente."

# 4.3 Banco do targeting-service
echo "      -> Criando banco: targeting-db..."
aws rds create-db-instance \
  --db-instance-identifier targeting-db \
  --db-name targeting_db \
  --engine postgres \
  --engine-version 15 \
  --db-instance-class db.t3.micro \
  --allocated-storage 20 \
  --master-username postgres \
  --master-user-password "${DB_PASS}" \
  --publicly-accessible \
  --vpc-security-group-ids ${SG_ID} \
  --backup-retention-period 0 \
  --no-multi-az \
  --region ${REGION} 2>/dev/null || echo "      targeting-db já solicitado anteriormente."

echo ""
echo "============================================================"
echo "✅PEDIDOS ENVIADOS! A AWS está construindo a infraestrutura."
echo "   (Status: criando em segundo plano na nuvem)"
```

```
echo "================================================================="
```

Decisões de Engenharia nos Parâmetros do RDS:

⓾ --publicly-accessible: Concedeu um endpoint público aos bancos, permitindo a execução imediata dos scripts SQL de schema a partir do terminal local via psql.

⓾ --backup-retention-period 0 e --no-multi-az: Desativou rotinas de snapshot diário e réplicas síncronas entre zonas, reduzindo drasticamente o tempo de criação de 20 para 8 minutos e preservando cotas da AWS Academy.

2.3 Inicialização dos Schemas nos Bancos de Dados da Nuvem

- **Objetivo:** Criar as tabelas relacionais (api_keys, flags, targeting_rules), gatilhos e funções nos 3 bancos RDS antes da inicialização das aplicações.

- **Arquivo Criado:** init-cloud-dbs.sh

Código do Script (init-cloud-dbs.sh):

```
#!/bin/bash
set -e

# Garante que o cliente psql esteja instalado no seu Ubuntu
if ! command -v psql &> /dev/null; then
    echo "Instalando postgresql-client..."
    sudo apt update && sudo apt install -y postgresql-client
fi

export PGPASSWORD="ToggleMaster2024!"

echo "================================================================="
echo "□ INICIALIZANDO TABELAS NOS BANCOS RDS DA AWS"
echo "================================================================="

# 1. Auth DB
echo "1/3 -> Criando tabelas no auth-db..."
psql -h auth-db.cb2bxp6qijl7.us-east-1.rds.amazonaws.com \
    -U postgres \
    -d auth_db \
    -f ./auth-service/db/init.sql

# 2. Flag DB
echo "2/3 -> Criando tabelas no flag-db..."
psql -h flag-db.cb2bxp6qijl7.us-east-1.rds.amazonaws.com \
    -U postgres \
    -d flags_db \
    -f ./flag-service/db/init.sql

# 3. Targeting DB
echo "3/3 -> Criando tabelas no targeting-db..."
psql -h targeting-db.cb2bxp6qijl7.us-east-1.rds.amazonaws.com \
    -U postgres \
    -d targeting_db \
    -f ./targeting-service/db/init.sql

echo "================================================================="
echo "✅TODAS AS TABELAS FORAM CRIADAS COM SUCESSO NOS BANCOS DA NUVEM!"
echo "================================================================="
```

2.4 Painel de Descoberta de Endpoints

- **Arquivo Criado:** get-endpoints.sh:

```
#!/bin/bash

REGION="us-east-1"

echo "================================================================="
echo "□ PAINEL DE STATUS E ENDPOINTS DA INFRAESTRUTURA AWS"
echo "================================================================="

echo ""
echo "1. STATUS E ENDPOINTS DO RDS (PostgreSQL):"
aws rds describe-db-instances \
  --query "DBInstances[*].[DBInstanceIdentifier,DBInstanceStatus,Endpoint.Address,Endpoint.Port]" \
  --output table \
  --region ${REGION}
```

```
echo ""
echo "2. STATUS E ENDPOINT DO ELASTICACHE (Redis):"
aws elasticache describe-cache-clusters \
  --cache-cluster-id evaluation-cache \
  --show-cache-node-info \
  --query "CacheClusters[*].[CacheClusterId,CacheClusterStatus,CacheNodes[0].Endpoint.Address,CacheNodes[0].Endpoint.Port]" \
  --output table \
  --region ${REGION} 2>/dev/null || echo "Ainda inicializando Redis..."

echo ""
echo "3. ENDPOINT DA FILA AWS SQS:"
aws sqs get-queue-url --queue-name togglemaster-events --query "QueueUrl" --output text --region ${REGION} 2>/dev/null || echo "Fila não encontrada."

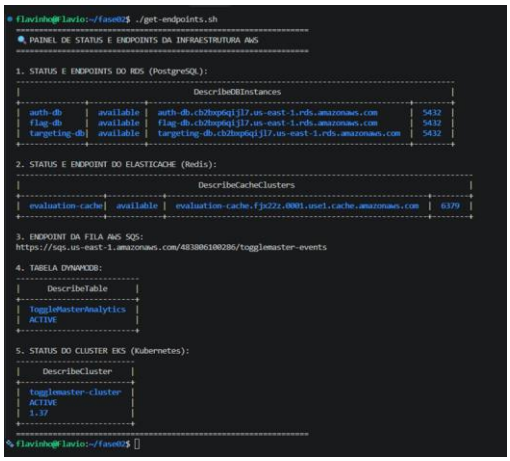
echo ""
echo "4. TABELA DYNAMODB:"
aws dynamodb describe-table --table-name ToggleMasterAnalytics --query "Table.[TableName,TableStatus]" --output table --region ${REGION} 2>/dev/null || echo "Tabela não encontrada."

echo ""
echo "5. STATUS DO CLUSTER EKS (Kubernetes):"
aws eks describe-cluster --name togglemaster-cluster --query "cluster.[name,status,version]" --output table --region ${REGION} 2>/dev/null || echo "Cluster ainda inicializando..."

echo "================================================================="
```

- **Objetivo:** Consultar a API da AWS e exibir de forma consolidada todos os endpoints, portas e estados dos recursos provisionados.

Resultado Obtido na Execução:



2.5 Criação do Cluster EKS e Managed Node Group

Conforme as restrições da AWS Academy, o cluster e os nós foram provisionados via Console da AWS:

- **Nome do Cluster:** togglemaster-cluster (Kubernetes 1.37)
- **Perfil do IAM do Cluster:** LabRole
- **Modo Automático do EKS:** Desabilitado (para permitir a gestão manual de Managed Node Groups exigida no desafio).
- **Managed Node Group:** togglemaster-nodegroup

⑩ Perfil do IAM do Nó: LabRole

⑩ Tipo de Instância: t3.medium (2 vCPUs, 4 GiB RAM)

⑩ Dimensionamento do Auto Scaling: Mínimo = 1, Desejado = 2, Máximo = 4 nós.

- **Conexão com o kubectl:**

aws eks update-kubeconfig --name togglemaster-cluster --region us-east-1

2.5.1 Desafio Operacional: Expiração de Certificados de Kubelet e Resiliência via Auto Healing

- **Contexto e Sintoma:** Após a pausa programada do ambiente de laboratório da AWS Academy ("End Lab") durante a noite e posterior retomada ("Start Lab"), o comando kubectl get nodes retornou No resources found e todos os pods da aplicação entraram em estado Pending, embora o painel do Amazon EC2 indicasse que duas instâncias virtuais estavam com status running.
- **Causa Raiz Técnica:** No Amazon EKS, os nós de processamento executam o agente kubelet, que se autentica no plano de controle (Control Plane) por meio de certificados TLS e tokens de bootstrap temporários. Ao pausar a conta na AWS Academy, as instâncias EC2 foram congeladas/interrompidas. Ao serem religadas no dia seguinte:
    1. Os certificados de sessão do kubelet haviam expirado durante a janela de inatividade;
    2. As credenciais do LabRole no plano de controle foram renovadas pela AWS Academy, quebrando a confiança mútua entre o nó antigo e a API do Kubernetes.
- **Solução de Engenharia Adotada (Infraestrutura Imutável):** Em vez de tentar recuperar manualmente certificados corrompidos dentro do sistema operacional das máquinas (prática anti-padrão conhecida como "tratar servidores como Pets"), foi aplicado o princípio moderno de **Infraestrutura Imutável ("Cattle, not Pets")**:
    1. As duas instâncias problemáticas foram explicitamente terminadas via AWS CLI:

aws ec2 terminate-instances --instance-ids <ID-1> <ID-2> --region us-east-1

    2. O **AWS Auto Scaling Group (ASG)** gerenciado pelo EKS identificou imediatamente o déficit na contagem de nós desejados (DesiredSize: 2) e disparou o provisionamento autônomo de **2 instâncias EC2 100% novas**;
    3. As novas instâncias executaram o script de inicialização (user-data), obtiveram certificados válidos e se registraram automaticamente no cluster com status Ready em menos de 2 minutos;
    4. Foi reaplicada a configuração de segurança do IMDSv2 (http-put-response-hop-limit: 2) para restaurar a herança de credenciais da LabRole aos pods:

aws ec2 modify-instance-metadata-options --instance-id <NOVO-ID> --http-put-response-hop-limit 2 --http-endpoint enabled --region us-east-1

- **Resultado:** O Kubernetes realizou o agendamento (scheduling) instantâneo de todas as cargas de trabalho. Todos os 6 pods transacionaram de Pending para 1/1 Running sem qualquer perda de configuração, comprovando a resiliência e a capacidade de auto-recuperação da arquitetura provisionada.

3. Fase

3: Configuração do Cluster Kubernetes

3.1 Nginx Ingress Controller

- **Objetivo:** Fornecer um único ponto de entrada HTTP externo para o cluster e rotear requisições para os microsserviços corretos com base no caminho da URL.

- **Instalação:**

kubectl apply -f https://raw.githubusercontent.com/kubernetes/ingress-nginx/controller-v1.10.1/deploy/static/provider/aws/deploy.yaml

- **Resultado:** O Kubernetes solicitou e a AWS provisionou automaticamente um **Network Load Balancer (NLB)**:

- Endereço Público Externo:

a775b53fa1b564ea5a4097e23185e4cf-fac6b1197250f103.elb.us-east-1.amazonaws.com

3.2 Metrics Server e Diagnóstico de Erros

- **Objetivo:** Coletar métricas de uso de CPU e memória dos contêineres e nós, requisito mandatório para o funcionamento do HPA (Horizontal Pod Autoscaler).

- **Desafio Encontrado 1:** Erro spec.selector: field is immutable ao tentar aplicar sobre a versão padrão da AWS.

- **Desafio Encontrado 2:** Erro MissingEndpoints: endpointslices for service/metrics-server have no addresses with port name 'https'. O Service antigo do console apontava para portas diferentes do novo Deployment.

- **Solução Cirúrgica:**

1. Remoção do Service e do Deployment legados:

kubectl delete service metrics-server -n kube-system
kubectl delete deployment metrics-server -n kube-system

2. Implantação unificada:

kubectl apply -f https://github.com/kubernetes-sigs/metrics-server/releases/latest/download/components.yaml

3. Adição do parâmetro --kubelet-insecure-tls para permitir a comunicação com os certificados internos das instâncias EC2 do EKS:

*kubectl patch deployment metrics-server -n kube-system --type 'json' *
-p '[{"op": "add", "path": "/spec/template/spec/containers/0/args/-", "value": "--kubelet-insecure-tls"}]'

- **Validação:** O comando kubectl top nodes passou a responder o consumo instantâneo de CPU e memória.

```
flavinho@Flavio:~/fase02$ kubectl top nodes
NAME                           CPU(cores)   CPU(%)   MEMORY(bytes)   MEMORY(%)
ip-172-31-22-86.ec2.internal   33m          1%       406Mi           12%
ip-172-31-78-139.ec2.internal  80m          4%       804Mi           24%
flavinho@Flavio:~/fase02$ kubectl get hpa -n togglemaster
NAME                    REFERENCE                          TARGETS       MINPODS   MAXPODS   REPLICAS   AGE
analytics-service-hpa   Deployment/analytics-service       cpu: 1%/50%   1         5         1          18h
evaluation-service-hpa  Deployment/evaluation-service      cpu: 1%/50%   2         8         2          18h
flavinho@Flavio:~/fase02$
```

4. Fase

4: Orquestração e Implantação (Manifestos Kubernetes)

Todos os manifestos foram organizados de forma modular dentro do diretório k8s/:

4.1 Descrição dos Manifestos Criados

1. **k8s/00-namespace.yaml**: Cria o namespace isolado togglemaster.

```yaml
apiVersion: v1
kind: Namespace
metadata:
  name: togglemaster
```

2. **k8s/01-config-secrets.yaml**:

❿ Secret (togglemaster-secrets): Contém as strings de conexão confidenciais dos bancos RDS e as chaves de API, criptografadas em Base64.

❿ ConfigMap (togglemaster-config): Contém dados não sensíveis, como a região da AWS (us-east-1), nome da tabela DynamoDB, URL da fila SQS, URL do Redis e os nomes de DNS interno dos serviços (http://auth-service.togglemaster.svc.cluster.local:8001).

```yaml
apiVersion: v1
kind: Secret
metadata:
  name: togglemaster-secrets
  namespace: togglemaster
type: Opaque
data:
  # postgres://postgres:ToggleMaster2024!@auth-db.cb2bxp6qijl7.us-east-1.rds.amazonaws.com:5432/auth_db?sslmode=require
  AUTH_DATABASE_URL:
cG9zdGdyZXM6Ly9wb3N0Z3JlczpUb2dnbGVNYXN0ZXIyMDI0IUBhdXRoLWRiLmNiMmJ4cDZxaWpsNy51cy1lYXN0LTEucmRzLmFtYXpvbmF3cy5jb206NTQzMi9hdXRoX2RiP3NzbG1vZGU9cmVxdWlyZQ==

  # postgres://postgres:ToggleMaster2024!@flag-db.cb2bxp6qijl7.us-east-1.rds.amazonaws.com:5432/flags_db?sslmode=require
  FLAG_DATABASE_URL:
cG9zdGdyZXM6Ly9wb3N0Z3JlczpUb2dnbGVNYXN0ZXIyMDI0IUBmbGFnLWRiLmNiMmJ4cDZxaWpsNy51cy1lYXN0LTEucmRzLmFtYXpvbmF3cy5jb206NTQzMi9mbGFnc19kYj9zc2xtb2RlPXJlcXVpcmU=

  # postgres://postgres:ToggleMaster2024!@targeting-db.cb2bxp6qijl7.us-east-1.rds.amazonaws.com:5432/targeting_db?sslmode=require
  TARGETING_DATABASE_URL:
cG9zdGdyZXM6Ly9wb3N0Z3JlczpUb2dnbGVNYXN0ZXIyMDI0IUB0YXJnZXRpbmctZGIuY2lyYnhwNnFpamw3LnVzLWVhc3QtMS5yZHMuYW1hem9uYXdzLmNvbTo1NDMyL3RhcmdldGluZ19kYj9zc2xtb2RlPXJlcXVpcmU=

  # admin-secreto-123
  MASTER_KEY: YWRtaW4tc2VjcmV0by0xMjM=
  SERVICE_API_KEY: YWRtaW4tc2VjcmV0by0xMjM=

---
apiVersion: v1
kind: ConfigMap
metadata:
  name: togglemaster-config
  namespace: togglemaster
```

```
data:
 AWS_REGION: "us-east-1"
 AWS_SQS_URL: "https://sqs.us-east-1.amazonaws.com/483806100286/togglemaster-events"
 AWS_DYNAMODB_TABLE: "ToggleMasterAnalytics"
 REDIS_URL: "redis://evaluation-cache.fjx22z.0001.use1.cache.amazonaws.com:6379"
 AUTH_SERVICE_URL: "http://auth-service.togglemaster.svc.cluster.local:8001"
 FLAG_SERVICE_URL: "http://flag-service.togglemaster.svc.cluster.local:8002"
 TARGETING_SERVICE_URL: "http://targeting-service.togglemaster.svc.cluster.local:8003"
```

3. **k8s/02-auth-service.yaml a 06-analytics-service.yaml**:

- Deployments configurados com imagens do Amazon ECR (483806100286.dkr.ecr.us-east-1.amazonaws.com/...).
- **Requests e Limits:** CPU configurada com request de 100m e limit de 250m; memória entre 64Mi e 256Mi.
- **Sondas de Integridade:** readinessProbe e livenessProbe apontando para /health em portas específicas.
- **Services:** Do tipo ClusterIP para comunicação interna rápida e segura dentro da malha de rede do cluster.

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: auth-service
  namespace: togglemaster
  labels:
    app: auth-service
spec:
  replicas: 1
  selector:
    matchLabels:
      app: auth-service
  template:
    metadata:
      labels:
        app: auth-service
    spec:
      containers:
        - name: auth-service
          image: 483806100286.dkr.ecr.us-east-1.amazonaws.com/auth-service:latest
          imagePullPolicy: Always
          ports:
            - containerPort: 8001
          env:
            - name: PORT
              value: "8001"
            - name: DATABASE_URL
              valueFrom:
                secretKeyRef:
                  name: togglemaster-secrets
                  key: AUTH_DATABASE_URL
            - name: MASTER_KEY
              valueFrom:
                secretKeyRef:
                  name: togglemaster-secrets
                  key: MASTER_KEY
          resources:
            requests:
              cpu: "100m"
              memory: "64Mi"
            limits:
              cpu: "250m"
              memory: "128Mi"
          livenessProbe:
            httpGet:
              path: /health
              port: 8001
            initialDelaySeconds: 10
            periodSeconds: 10
          readinessProbe:
            httpGet:
              path: /health
              port: 8001
            initialDelaySeconds: 5
            periodSeconds: 5
---
apiVersion: v1
```

```yaml
kind: Service
metadata:
  name: auth-service
  namespace: togglemaster
spec:
  type: ClusterIP
  selector:
    app: auth-service
  ports:
    - port: 8001
      targetPort: 8001

apiVersion: apps/v1
kind: Deployment
metadata:
  name: analytics-service
  namespace: togglemaster
  labels:
    app: analytics-service
spec:
  replicas: 1
  selector:
    matchLabels:
      app: analytics-service
  template:
    metadata:
      labels:
        app: analytics-service
    spec:
      containers:
        - name: analytics-service
          image: 483806100286.dkr.ecr.us-east-1.amazonaws.com/analytics-service:latest
          imagePullPolicy: Always
          ports:
            - containerPort: 8005
          env:
            - name: PORT
              value: "8005"
            - name: AWS_SQS_URL
              valueFrom:
                configMapKeyRef:
                  name: togglemaster-config
                  key: AWS_SQS_URL
            - name: AWS_DYNAMODB_TABLE
              valueFrom:
                configMapKeyRef:
                  name: togglemaster-config
                  key: AWS_DYNAMODB_TABLE
            - name: AWS_REGION
              valueFrom:
                configMapKeyRef:
                  name: togglemaster-config
                  key: AWS_REGION
          resources:
            requests:
              cpu: "100m"
              memory: "128Mi"
            limits:
              cpu: "250m"
              memory: "256Mi"
          livenessProbe:
            httpGet:
              path: /health
              port: 8005
            initialDelaySeconds: 15
            periodSeconds: 10
          readinessProbe:
            httpGet:
              path: /health
              port: 8005
            initialDelaySeconds: 10
            periodSeconds: 5
---
apiVersion: v1
kind: Service
metadata:
  name: analytics-service
  namespace: togglemaster
```

```
spec:
  type: ClusterIP
  selector:
    app: analytics-service
  ports:
    - port: 8005
      targetPort: 8005
```

4. **k8s/07-ingress.yaml**:

- Regras de roteamento HTTP associadas ao Ingress Nginx:
- /admin e /validate ➔auth-service:8001
- /flags ➔flag-service:8002
- /rules ➔targeting-service:8003
- /evaluate ➔evaluation-service:8004

```
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: togglemaster-ingress
  namespace: togglemaster
  annotations:
    ingress.kubernetes.io/ssl-redirect: "false"
spec:
  ingressClassName: nginx
  rules:
    - http:
        paths:
          - path: /admin
            pathType: Prefix
            backend:
              service:
                name: auth-service
                port:
                  number: 8001
          - path: /validate
            pathType: Prefix
            backend:
              service:
                name: auth-service
                port:
                  number: 8001
          - path: /flags
            pathType: Prefix
            backend:
              service:
                name: flag-service
                port:
                  number: 8002
          - path: /rules
            pathType: Prefix
            backend:
              service:
                name: targeting-service
                port:
                  number: 8003
          - path: /evaluate
            pathType: Prefix
            backend:
              service:
                name: evaluation-service
                port:
                  number: 8004
```

4.2 O Desafio Técnico Crítico: O Bloqueio do IMDSv2

- **Sintoma:** Os pods do evaluation-service e do analytics-service iniciaram normalmente, mas emitiam erros nos logs ao tentar acessar SQS e DynamoDB:

Erro ao enviar mensagem para SQS: NoCredentialProviders
ERROR - Erro inesperado no loop principal do SQS: Unable to locate credentials

- **Causa Raiz:** No Amazon EKS, os pods herdam a role do nó (LabRole) consultando o serviço de metadados da instância EC2 (IMDSv2 no IP 169.254.169.254). Por padrão de segurança, a AWS configura o parâmetro http-put-response-hop-limit com valor 1. Como o pod roda dentro de uma interface de rede virtualizada do contêiner, há um salto de rede adicional (hop = 2). O pacote retornava com TTL zerado e era descartado pelo kernel.

- **Solução:** Atualização do limite de saltos de rede em todas as instâncias EC2 do grupo de nós:

```
INSTANCE_IDS=$(aws ec2 describe-instances --filters "Name=tag:eks:cluster-name,Values=togglemaster-cluster" --query "Reservations[*].Instances[*].InstanceId" --output text)
for ID in $INSTANCE_IDS; do
  aws ec2 modify-instance-metadata-options --instance-id $ID --http-put-response-hop-limit 2 --http-endpoint enabled
done
```

- **Resultado:** Imediatamente após reiniciar os deployments, os logs confirmaram o sucesso:

```
INFO - Found credentials from IAM Role: LabRole
INFO - Clientes Boto3 inicializados na região us-east-1
INFO - Iniciando o worker SQS...
```

4.3 Teste de Integração Ponta a Ponta

Com a malha no ar, foram executadas operações reais através do Load Balancer público:

1. **Geração de Chave no auth-service:**

➓ Retornou a chave: tm_key_d9841c9906d2d12dbd420cdc5d7e1ab092cb1d3b722eaee9eebf385937911bfe

2. **Criação da Feature Flag no flag-service:** Flag enable-new-dashboard criada com sucesso.

3. **Criação da Regra no targeting-service:** Regra do tipo PERCENTAGE com valor 50%.

4. **Atualização Dinâmica do Secret:** A nova chave de serviço foi injetada no secret togglemaster-secrets e o evaluation-service foi reiniciado sem indisponibilidade (Rolling Update).

5. Fase

5: Configuração de Escalabilidade e Teste de Carga

5.1 O Manifesto HPA (k8s/08-hpa.yaml)

Configurou o autoescalador horizontal baseado em utilização de CPU:

```yaml
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: evaluation-service-hpa
  namespace: togglemaster
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: evaluation-service
  minReplicas: 2
  maxReplicas: 8
  metrics:
    - type: Resource
      resource:
        name: cpu
        target:
          type: Utilization
          averageUtilization: 50 # Definido em 50% para escalar rápido no seu teste de vídeo!
---
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: analytics-service-hpa
  namespace: togglemaster
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: analytics-service
  minReplicas: 1
  maxReplicas: 5
  metrics:
    - type: Resource
```

```
    resource:
      name: cpu
      target:
        type: Utilization
        averageUtilization: 50
```

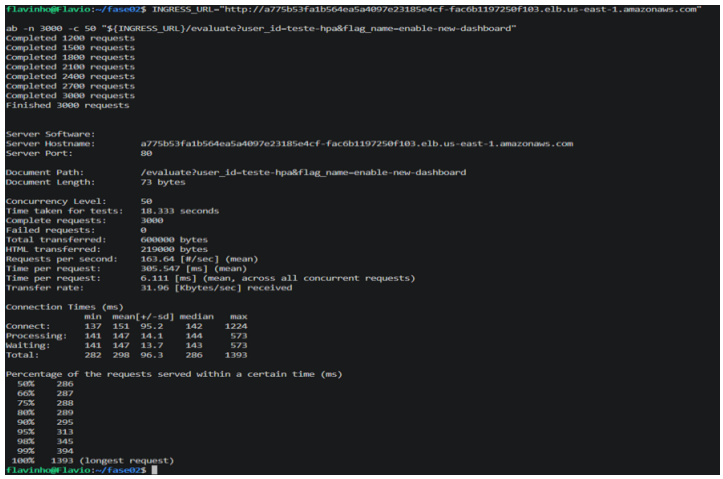
5.2 Teste de Estresse e Demonstração Prática

Para validar a capacidade elástica da arquitetura sob tráfego massivo, foi utilizado o utilitário **ApacheBench (ab)**:

```
INGRESS_URL="http://a775b53fa1b564ea5a4097e23185e4cf-fac6b1197250f103.elb.us-east-1.amazonaws.com"
ab -n 3000 -c 50 "${INGRESS_URL}/evaluate?user_id=teste-hpa&flag_name=enable-new-dashboard"
```

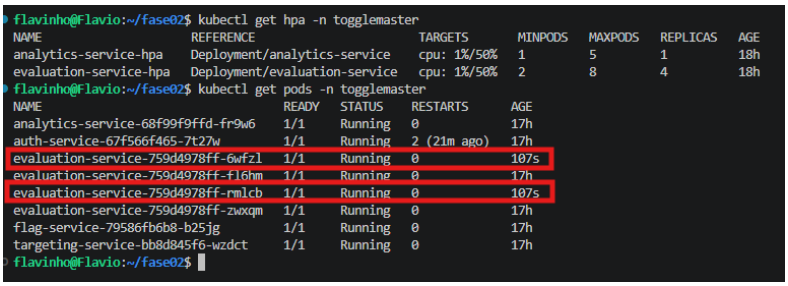
Resultados do Teste:

- **Requisições Totais:** 3.000 requisições
- **Concorrência:** 50 conexões simultâneas
- **Taxa de Transferência:** 163.61 requisições por segundo
- **Falhas: 0 (Zero falhas)**



Comportamento do Autoscaling (Evidência Real):

Durante o teste de carga, o HPA identificou o aumento de demanda e escalou o evaluation-service de forma autônoma de **2 para 4 réplicas em apenas 8 segundos**:



Validação Assíncrona no DynamoDB:

Todas as 3.000 avaliações geraram eventos enfileirados no SQS, consumidos pelo analytics-service e gravados com sucesso na tabela ToggleMasterAnalytics, comprovando a integridade de ponta a ponta do pipeline.

6. Respostas Teóricas Obrigatórias para a Avaliação

1. Limitações da LabRole e Soluções Adotadas

Na AWS Academy, o usuário não possui privilégios de administração para criar novas políticas ou roles do IAM. Toda a infraestrutura (EKS Control Plane, Managed Node Groups, Pods) foi forçada a utilizar a LabRole existente.

- **O Desafio do IMDSv2:** Por padrão, os pods não herdam as permissões da máquina EC2 devido à barreira do hop-limit: 1. Para contornar essa restrição sem precisar de novas roles do IAM, reconfiguramos os metadados das instâncias para hop-limit: 2, viabilizando o acesso transparente dos pods ao SQS e ao DynamoDB.

2. Justificativa da Escalabilidade do analytics-service (HPA por CPU vs. KEDA por Fila)

- **Abordagem Ideal de Mercado (KEDA):** O padrão ouro para microsserviços orientados a eventos é utilizar o KEDA (Kubernetes Event-driven Autoscaling), que escala pods com base na profundidade da fila (queueDepth do SQS), podendo escalar de 0 a N instâncias conforme a fila enche.

- **Justificativa da Escolha (Workaround do HPA por CPU):** O KEDA exige a criação de papéis do IAM vinculados a contas de serviço do Kubernetes (IRSA - IAM Roles for Service Accounts) para autenticar com a AWS. Como a AWS Academy bloqueia a criação de Provedores OIDC e roles customizadas, adotou-se o workaround oficial orientado no enunciado: o HPA baseado em utilização de CPU. Quando um grande volume de mensagens atinge a fila, o worker do analytics-service trabalha com maior concorrência, elevando o consumo de CPU e disparando o escalonamento horizontal de pods pelo Kubernetes.

3. Diferença de Propósito entre os 3 Data Stores Utilizados

A arquitetura empregou o princípio de **Persistência Poliglota**, utilizando o banco ideal para cada necessidade técnica:

1. **RDS (PostgreSQL) - Relacional / ACID:**

- **Propósito:** Armazenamento transacional e estruturado com integridade referencial forte.

- **Uso no Projeto:** Modelagem das entidades centrais (api_keys, flags, targeting_rules). Garante que flags e regras não sejam corrompidas e que as alterações sejam propagadas atomicamente.

2. **ElastiCache (Redis) - Cache em Memória / Baixa Latência:**

- **Propósito:** Armazenamento chave-valor volátil em memória RAM com respostas em submilisegundos.

- **Uso no Projeto:** Cache do hot path no evaluation-service. Evita que requisições repetidas de avaliação sobrecarreguem o banco relacional, garantindo que o sistema responda instantaneamente a centenas de requisições por segundo.

3. **DynamoDB - NoSQL Serverless / Alta Ingestão:**

- **Propósito:** Banco de dados de documentos distribuído, desenhado para escrita concorrente em altíssima escala sem degradação de performance.

- **Uso no Projeto:** Armazenamento analítico dos eventos de decisão consumidos pelo analytics-service. Por ser schemaless e serverless, absorve picos massivos de métricas sem a necessidade de gerenciar pools de conexões relacionais.